

LineageRCA — Counterfactual Replay for Data-Pipeline Root Cause Analysis

Sep 30, 2026 · @Wolf

When a data-quality alarm fires, we don't just guess the upstream cause from the lineage graph. We re-run the pipeline on the table's earlier snapshot and check whether the problem goes away.

The problem in plain terms

A dashboard number suddenly looks wrong, and the data passed through many steps to get there. Engineers today look at the lineage map, see which upstream tables changed recently, and pick the most likely culprit. That is a guess based on "these two things happened near each other". It is not proof.

Our idea is to test the guess. Take the suspected upstream table, roll it back to how it looked before the suspected change (lakehouse tables like Iceberg and Delta keep old snapshots, called time travel), run the downstream step again, and compare. If the bad output disappears, that table was the cause. If it doesn't, rule it out and try the next suspect.

This is a *counterfactual* test: "what would the output have been if this table had not changed?"

Why it matters now

The market leader stops at a suggestion. Databricks' Unity Catalog uses lineage to point at "the most likely reason" for a data-quality issue, such as a failed job or an unhealthy upstream table. It does not re-run anything to confirm the suggestion ([Databricks docs](#), updated Sept 2026).

Practitioners have named the gap. A Feb 2026 essay lists "time as a non-first-class dimension" as one of 13 gaps that break root-cause analysis in lineage tools, but only proposes better forensic metadata for humans to read ([Hash Block, Medium](#)). A Marquez user asked how to query lineage at a historical version ([Marquez #3065](#), title only).

Be honest about the evidence: this is real but indirect. We found no issue or postmortem that says "I want to replay against an old snapshot to confirm a root cause". The dossier scores real-world grounding 2/5, and we agree.

What we're building

A small system with four parts. A pipeline of 3 to 4 Iceberg or Delta tables produces data. A lineage layer records which table feeds which. A data-quality monitor flags an anomaly, such as a row-count drop. A replay harness then investigates.

When the monitor fires, the harness walks the lineage graph backward and lists upstream tables that changed shortly before the anomaly. For each suspect, it re-runs the downstream step on that table's pre-change snapshot and compares the result with today's output. Each suspect gets a verdict: **CONFIRMED** (the anomaly disappears) or **DENIED** (it stays).

Claim we test: replay-confirmation finds the true cause with higher precision than lineage correlation alone, on a multi-stage pipeline with injected faults.

Narrowed version (the novel part): using native lakehouse time travel as the replay mechanism, guided by lineage shortlisting, to confirm or deny data-quality root causes.

Where this fits our allowed domains

This sits between data engineering and DevOps, with a cloud angle. Please confirm this mapping with the mentor, since it is our reading of the domain list.

- **Data engineering:** the object under study is a lakehouse pipeline, its tables, snapshots and lineage.
- **DevOps (DataOps):** the system works like an incident-response tool for data pipelines. It answers "which change broke this table?" the way a CI/CD tool answers "which commit broke this build?"
- **Cloud:** Iceberg and Delta tables normally live on cloud object storage, so the design carries over to a real cloud setup.

Why we focus on replay, not just lineage

Most existing approaches either guess or prove on paper. The few that do replay use a different substrate; the novelty section covers them (Newt, aaiclick, BugDoc).

Approach	How it decides the cause	Gap for us
Lineage correlation (Databricks Unity Catalog, IBM patent US10776740B2)	Compares lineage graphs and recent changes, then suggests a likely cause	Suggestion only. Nothing re-runs to confirm it.
Heuristic inference (patent US11960471)	Infers data quality from lineage plus ML, without touching the data	No replay, no time travel.
Algebraic counterfactuals (Meliou and Suciu, VLDB 2014)	Removes a tuple on paper and solves with SAT	A thought experiment, not a re-run of a real pipeline.
Parameter replay (BugDoc, SIGMOD 2020)	Re-runs a pipeline with different parameters to find the failing setting	Varies parameters, not table snapshots. Not lineage-driven.
LLM-based ETL diagnosis (ACM, 2026)	An LLM reasons over logs and lineage to suggest causes and fixes	Inference, not replay.
LineageRCA (ours)	Re-runs downstream steps on the suspect table's earlier snapshot and compares	The idea to test.

What we're actually analyzing

Everything comes from our own test pipeline, so no real company data is needed.

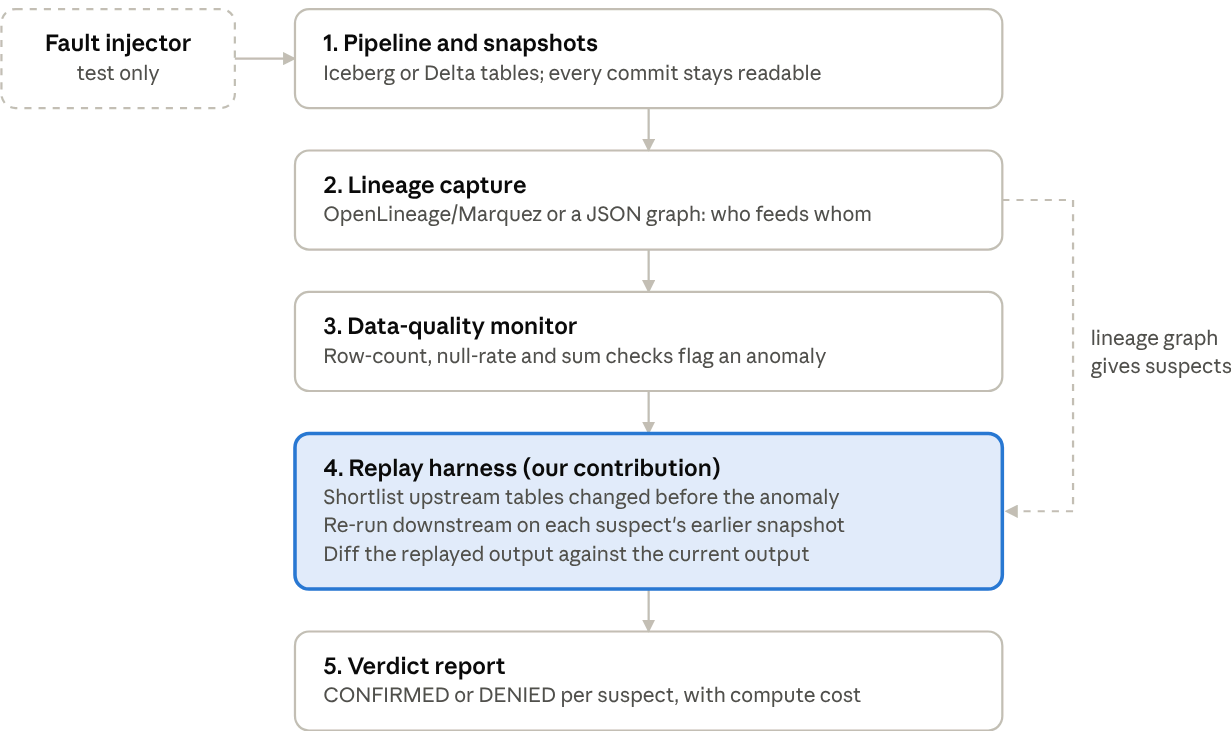
Input	Status in this project
Iceberg or Delta table snapshots (each commit is an immutable version)	Primary. The raw material that replay reads.
Lineage events (OpenLineage/Marquez, or a hand-built JSON graph)	Primary. Says which tables feed which, to shortlist suspects.
Data-quality metrics (row count, null rate, aggregate sums)	Primary. Trigger the investigation.

Input	Status in this project
Transform code of each pipeline stage	Primary. Must be re-runnable on any snapshot.
Injected faults (bad join key, dropped filter, type coercion bug)	Ground truth for scoring the method.
Streaming, incremental or side-effecting transforms	Out of scope for Phase I. Replay is not clean for these.
Real production pipelines	Out of scope. Needs company access.

Proposed architecture

A five-stage pipeline, kept small enough for a four-person team. The replay harness is the part that is new.

Replay turns a lineage guess into a tested verdict



LineageRCA pipeline · 5 stages, replay harness highlighted

The dashed line shows the replay harness taking its suspect list from the lineage graph, while the monitor tells it when to start.

Tools used in each layer

Layer	Tool(s)	Notes
Data generator	Faker-style scripts; optionally a TPC-H/TPC-DS subset or NYC taxi trips	Makes the raw table and scripts the incidents. No licensing barrier.
Pipeline and tables	PySpark local mode, or DuckDB with pyiceberg; Iceberg or Delta tables	Runs on a laptop. Each commit becomes a snapshot we can read later.
Lineage	OpenLineage events stored in Marquez, or a hand-built JSON column-lineage graph	Hand-built is the fallback if OpenLineage setup eats too much time.
Monitor	Python checks on row-count, null-rate and aggregate-sum change	Simple and explainable. Flags the anomaly that starts an investigation.
Replay harness	Python driver calling time-travel reads (Iceberg snapshot ID, Delta <code>VERSION AS OF</code>)	The core contribution. Re-runs the downstream stage and diffs the outputs.
Report	Notebook or small dashboard	Shows CONFIRMED or DENIED per suspect, plus compute cost per incident.

What kinds of failures we can work on

The first three come from the dossier's scripted incidents. The last two are the boundary we test in Phase II.

Category	What goes wrong	How replay should behave
Bad join key change	An upstream table changes its key, so a downstream join drops or duplicates rows	Replay on the pre-change snapshot restores the row count: CONFIRMED.
Dropped filter	A filter step is removed, so extra rows flow downstream	Replay on the earlier snapshot removes the surplus: CONFIRMED.

Category	What goes wrong	How replay should behave
Schema or type coercion bug	A column type changes and values silently turn into nulls or zeros	Replay on the earlier snapshot restores the values: CONFIRMED .
Two causes at once	Two upstream tables changed and both contribute	Replaying one suspect alone may only partly fix the output. Tests the method's limits.
Non-deterministic or stateful transform	Output depends on wall-clock time, external calls or incremental state	Replay may differ even with no real cause. The known breaking point (research question 2).

Root-cause techniques and why we chose ours

Method	How it works (brief)	Role in this project
Lineage shortlisting	Walk the lineage graph backward from the anomalous table. Keep upstream tables changed within a set time window.	Step 1. Keeps the number of replays small (about 5 suspects per incident).
Time-travel replay and diff	Re-run the downstream stage on the suspect's pre-change snapshot. Diff the result against the current output.	Step 2, the core. The verdict is CONFIRMED if the anomaly disappears, else DENIED .
Repeated replay	Run each replay 3 times to average out incidental non-determinism.	Adds a cheap guard against noisy transforms. About 30 minutes of compute per incident.
Lineage-correlation only	Rank suspects by graph position and recency, with no re-run.	The baseline we must beat (the Databricks and IBM-patent pattern).
Parameter replay (BugDoc-style)	Vary pipeline parameters instead of table snapshots.	A second baseline. Shows whether the snapshot substrate matters.

Method	How it works (brief)	Role in this project
LLM-based diagnosis	An LLM reads logs and lineage and proposes a cause.	Possible later add-on for ranking suspects. Not needed for the core claim.

Why this idea is worth pursuing: novelty assessment

The general idea of "replay to confirm a cause" is not new, so we do not claim it. What we found open is the specific combination: lineage shortlisting plus replay on native lakehouse time-travel snapshots, aimed at data-quality root causes, and evaluated properly. This is "open as far as we found", not "proven empty". It rests on about 30 searches for the dossier plus our own follow-up checks, and Google Scholar was not part of the dossier's search.

Closest related work:

- [BugDoc](#) (SIGMOD 2020): real counterfactual replay, but it varies pipeline parameters, not table snapshots, and is not lineage-driven.
- [Newt](#) (De, 2012): lineage-based replay and debugging for Hadoop-style data-processing systems. It predates lakehouses, so it uses no Iceberg or Delta time travel. The original dossier missed it; we must cite it.
- [aaiclick](#): an open-source AI-agent lineage debugger. Its "Tier 2" re-runs the whole pipeline with all intermediate tables kept, to find where data diverged. It re-executes rather than reading a past snapshot, and it is a small project. The dossier missed it.
- [whydunit](#) (v0.1.1, July 2026): counterfactual replay to diagnose LLM pipeline failures. Different domain, no lineage.
- Root-Cause Diagnosis with Remediation Recommendations for ETL Pipelines (ACM, 2026, DOI 10.1145/3788853.3801613): the dossier could not read it. It turns out to be LLM-reasoning based, not replay.
- IBM patent US10776740B2 and patent US11960471: lineage-graph diffing and heuristic inference, no replay.
- [Meliou and Suci, VLDB 2014](#): counterfactuals defined algebraically on paper, not by re-running a pipeline.
- Databricks Unity Catalog: lineage-based suggestion of the likely cause, no replay.

Framing for the mentor: our contribution is not inventing replay. It is showing that native time travel makes replay cheap and practical inside a lineage-guided workflow, and measuring how much it beats lineage correlation, including where it breaks (non-deterministic transforms). This is a narrower claim than the dossier's "novelty 4/5", so we score it conservatively and expect a workshop-level paper, not a top-tier one.

The state of existing tools and benchmarks

Tool or benchmark	What it has	What it lacks for us
Databricks Unity Catalog (lineage plus Lakehouse Monitoring)	Anomaly detection and lineage-based "most likely reason"	Suggestion only. Closed product, so we can't study or extend it.
OpenLineage and Marquez	Open standard and server for lineage events	Records lineage. Does no root-cause confirmation.
Iceberg and Delta time travel	Immutable snapshots readable by ID or version	A storage feature. Nobody has packaged it as a replay engine for root-cause checks that we found.
BugDoc	Open research system for parameter replay	Wrong substrate (parameters, not snapshots).
aaiclick	Lineage debugging with full pipeline re-run	Small AI-agent tool. No snapshot replay, no evaluation against a lineage baseline.
Delta time travel for monitoring (Medium, 2025)	Snapshot diffs for drift monitoring	Drift monitoring only. Explicitly not root-cause replay.
Conclusion	We found no public benchmark of injected-fault incidents on lakehouse pipelines with known ground-truth causes	We plan to build a small labeled scenario suite as the first deliverable.

Building our own test environment

We have no real company pipeline, so we build a synthetic one and break it on purpose. The key rule is that Phase I transforms must be pure and deterministic (same inputs, same output), so a replay is a fair test. We state this as a scoping decision, not an oversight.

- Build a 3 to 4 stage pipeline such as `raw_orders` to `cleaned_orders` to `daily_revenue_agg`, with each stage committed as Iceberg or Delta snapshots.
- Script 20 fault scenarios (bad join key, dropped filter, type coercion bug and so on). Each one becomes a new snapshot with a known ground-truth cause.
- Capture lineage for every run, and record the monitor's anomaly flag.
- Run the replay harness on each incident and log every verdict and its compute time.
- Hold out some scenario types (not just instances) from the ones used to tune thresholds, so we don't score ourselves on tests we designed the method around.

Compute estimate: 5 suspects x 3 replays x about 2 minutes is about 30 minutes per incident, so about 10 hours for all 20 scenarios. That fits an overnight laptop run or a few dollars of cloud credit.

What the final dataset looks like

Three linked tables hold everything the evaluation needs: one row per table snapshot, one row per lineage edge, and one row per replay verdict.

1. Snapshot table

Field	Description
snapshot_id	Iceberg snapshot ID or Delta version number.
table_name	Which pipeline table this snapshot belongs to.
committed_at	When the snapshot was committed.
change_type	Normal load, or which injected fault (if any) it contains.
row_count	Rows at this snapshot, used by the monitor.
metric_value	Key aggregate at this snapshot, such as daily revenue.

2. Lineage edge table

Field	Description
upstream_table	Table the data came from.
downstream_table	Table derived from it.
transform_id	Which pipeline stage does the derivation.
run_id	Which pipeline run emitted this edge.
lineage_source	OpenLineage event or hand-built graph.

3. Replay verdict table

Field	Description
incident_id	Unique ID of the detected anomaly.
suspect_table	Upstream table being tested.
suspect_snapshot_id	Pre-change snapshot used for the replay.
replay_runs	How many repetitions were run (planned: 3).

Field	Description
anomaly_cleared	Whether the replayed output removed the anomaly.
verdict	CONFIRMED or DENIED.
ground_truth_cause	The injected fault, used only to score the verdict.
compute_seconds	Cost of the replay, to check it stays practical.

How this meets the five evaluation criteria

Criterion	How this idea meets it
Complexity and depth	Spans lakehouse table formats, lineage capture, anomaly detection, and counterfactual replay design.
Feasibility	A toy pipeline on a laptop with synthetic data. About 10 hours of total replay compute for 20 scenarios. The strongest feasibility in the dossier's candidate set (4 of 5).
Relevance and impact	Data teams do root-cause analysis by hand today. Practitioners name the point-in-time gap, but direct demand for replay is not yet documented (grounding 2 of 5).
Innovation	The replay-plus-time-travel-plus-lineage combination was not found published. Related replay work exists (BugDoc, Newt, aaiclick), so the claim is narrow.
Long-term value	Natural arc: prototype, evaluation on a semi-realistic DAG, then non-deterministic and incremental transforms. Publication is plausible at workshop level.

Where it is weakest

- **Replay validity:** if a transform reads the clock, calls an outside service, or keeps incremental state, replay is not a clean counterfactual. This is the biggest technical risk. We constrain Phase I to pure, deterministic, batch transforms and report the limit openly.
- **Circular evaluation:** injecting a fault and checking whether replay finds it tests our own assumptions. We hold out fault types, and we say the result is synthetic-only.
- **Publishability:** a top-tier venue would need a more realistic evaluation and stronger baselines than one semester can give (dossier score 2 of 5).

Four-semester plan

This is our proposed arc, not something fixed by the dossier. It mirrors the CascadeGuard plan so the two can be compared.

Semester	Focus	Key deliverable
Sem 5	Build the toy 3 to 4 stage pipeline, lineage capture, monitor, and a first replay harness. Run a handful of injected faults.	Working end-to-end demo: an anomaly is flagged and the harness returns CONFIRMED for the true cause.
Sem 6	Complete the 20-scenario suite. Compare replay with lineage-only and BugDoc-style baselines.	Scenario benchmark and first precision/recall results.
Sem 7	Stress the method: add mild non-determinism, incremental steps, and a TPC-H-derived DAG. Find the breaking point.	Limits analysis and semi-realistic evaluation.
Sem 8	Write-up and submission.	Workshop or student-conference paper plus final report.